



Chapter 13:

Abstract Class and Interface

13.1. Introduction

👉 Introduction

- ✨ In OOP, we often need to define classes that share common behavior but may have different implementations.
- ✨ In Java, **abstract class** and **interface** help us create a common structure and behavior that related classes can share.

Example:

- ✨ Consider a scenario where we have different types of vehicles (e.g., Car, Truck, Motorcycle) that share some common behavior (e.g., start, stop) but have different implementations.
- ✨ We can define an abstract class `Vehicle` with abstract methods `start()` and `stop()`, and then create subclasses like `Car`, `Truck`, and `Motorcycle` that provide their own implementations of these methods.

13.2. Abstract Class

👉 Motivation

✨ In OOP, we have to implement methods in the SuperClass even if it doesn't work correctly. This can lead to code that is not reusable or maintainable.

Example: Concrete class `Vehicle` with a method `isRunning()` .

```
public class Vehicle {  
    public boolean isRunning() {  
        return false; // must return true or false, but doesn't work correctly  
    }  
}
```

Explain: The `Vehicle` class is a regular class, so you can create objects from it, even if it doesn't work correctly.

✨ This is where **abstract class** come into play, allowing us to define a common structure and behavior for subclasses.

👉 What is Abstract Class?

- ✨ An **Abstract Class** is a class you cannot create objects from directly.
- ✨ An abstract class can contain both concrete methods (with implementation) and abstract methods (without implementation).
 - ✨ **Concrete Methods:** Methods with a body that can provide default behavior.
 - ✨ **Abstract Methods:** Methods declared without a body, which must be implemented by subclasses.

👉 Create an Abstract Class

✨ An **abstract class** is defined using the `abstract` keyword. It can have abstract methods that must be implemented by any subclass.

Syntax:

```
public abstract class AbstractClassName {  
    // Abstract method (no implementation)  
    public abstract void abstractMethod();  
  
    // Concrete method (with implementation)  
    public void concreteMethod() {  
        // Implementation code  
    }  
}
```

👉 Using an Abstract Class

✨ To inherit from an abstract class, a subclass must use the `extends` keyword and provide implementations for all abstract methods.

Syntax:

```
public class SubClassName extends AbstractClassName {}
```

Explain:

✨ The subclass `SubClassName` inherits from the abstract class `AbstractClassName` and must implement all abstract methods defined in the abstract class.

✨ If the subclass does not implement all abstract methods, it must also be declared as abstract.

👉 **Abstract Class Types**

✨ Abstract classes can be categorized into different types based on their structure and purpose.

✨ Here are some common types of abstract classes:

- ✨ **Pure Abstract Class**

- ✨ **Partial Abstract Class**

- ✨ **Root Abstract Class**

- ✨ **Intermediate Abstract Class**

- ✨ **Template Method Abstract Class**

- ✨ **Helper Abstract Class**

- ✨ **Stateful Abstract Class**

- ✨ **Stateless Abstract Class**

👉 Pure Abstract Class

- ✨ A **Pure Abstract Class** contains only abstract methods and no concrete methods. It serves as a blueprint for subclasses to implement.
- ✨ It cannot be instantiated directly and is used to define a common interface for subclasses.

Syntax: Pure Abstract Class

```
public abstract class AbstractClassName {  
    // Abstract method (no implementation)  
    public abstract void abstractMethod();  
}
```

Example: Pure Abstract Class

```
abstract class A {  
    public abstract void methodA(); // Abstract method  
}  
class B extends A {  
    @Override  
    public void methodA() {  
        System.out.println("Implementation of methodA in class B");  
    }  
}  
public class Test {  
    public static void main(String[] args) {  
        // A a = new A(); // This line would cause an error because A is abstract  
        B b = new B(); // Create an instance of B  
        b.methodA(); // Call the implemented method  
    }  
}
```

👉 Partial Abstract Class

✨ A **Partial Abstract Class** contains both abstract methods and concrete methods. It can provide some default behavior while still requiring subclasses to implement certain methods.

✨ This allows for a mix of shared functionality and specific behavior that subclasses must define.

Syntax: Partial Abstract Class

```
public abstract class AbstractClassName {  
    // Abstract method (no implementation)  
    public abstract void abstractMethod();  
  
    // Concrete method (with implementation)  
    public void concreteMethod() {}  
}
```

Example: Partial Abstract Class

```
abstract class A {  
    public abstract void methodA(); // Abstract method  
    public void methodB() { // Concrete method  
        System.out.println("Default methodB");  
    }  
}  
  
class B extends A {  
    public void methodA() { // Implementing the abstract method  
        System.out.println("B's methodA");  
    }  
}  
  
public class Test {  
    public static void main(String[] args) {  
        B b = new B();  
        b.methodA(); // Output: B's methodA  
        b.methodB(); // Output: Default methodB  
    }  
}
```

👉 Root Abstract Class

✨ A **Root Abstract Class** serves as a base for other abstract classes, providing common functionality that can be shared among subclasses.

Syntax: Root Abstract Class

```
public abstract class RootAbstractClass {  
    // Abstract method (no implementation)  
    public abstract void rootMethod();  
  
    // Concrete method (with implementation)  
    public void concreteMethod() {}  
}
```

👉 Intermediate Abstract Class

- ✨ An **Intermediate Abstract Class** extends a root abstract class and adds additional abstract methods or concrete methods. It serves as an intermediate layer in the class hierarchy.
- ✨ This allows for further specialization of behavior while still adhering to the structure defined by the root abstract class.

Syntax: Intermediate Abstract Class

```
public abstract class IntermediateAbstractClass extends RootAbstractClass {  
    // Abstract method (no implementation)  
    public abstract void intermediateMethod();  
  
    // Concrete method (with implementation)  
    public void concreteMethod() {}  
}
```

👉 **Template Method Abstract Class**

✨ A **Template Method Abstract Class** defines a template method that outlines the steps of an algorithm, allowing subclasses to implement specific steps while keeping the overall structure intact.

Syntax: Template Method Abstract Class

```
public abstract class TemplateMethodAbstractClass {  
    // Template method  
    public final void templateMethod() {  
        stepOne();  
        stepTwo();  
    }  
  
    // Abstract steps to be implemented by subclasses  
    protected abstract void stepOne();  
    protected abstract void stepTwo();  
}
```


👉 Helper Abstract Class

✨ A **Helper Abstract Class** provides static utility methods that can be used without instantiating the class. It may also contain abstract methods that subclasses must implement.

Syntax: Helper Abstract Class

```
public abstract class HelperAbstractClass {  
    // Static utility method  
    public static void helperMethod() {}  
  
    // Abstract method (no implementation)  
    public abstract void abstractMethod();  
}
```

👉 **Stateful Abstract Class**

✨ A **Stateful Abstract Class** maintains state through instance variables and provides methods to manipulate that state.

Syntax: Stateful Abstract Class

```
public abstract class StatefulAbstractClass {  
    // Instance variable to maintain state  
    protected int state;  
  
    // Abstract method (no implementation)  
    public abstract void updateState(int newState);  
  
    // Concrete method (with implementation)  
    public void displayState() {  
        System.out.println("Current state: " + state);  
    }  
}
```

Stateless Abstract Class

✨ A **Stateless Abstract Class** does not maintain any instance variables and provides only static utility methods or abstract methods.

Syntax: Stateless Abstract Class

```
public abstract class StatelessAbstractClass {  
    // Static utility method  
    public static void utilityMethod() {}  
  
    // Abstract method (no implementation)  
    public abstract void performAction();  
}
```

Abstract Class Rules

- ✨ An abstract class cannot be instantiated directly.
- ✨ It can contain abstract methods, which must be implemented by any non-abstract subclass.
- ✨ It can also contain concrete methods with implementation.
- ✨ An abstract class can have constructors, fields, and methods like a regular class.
- ✨ An abstract class can extend another class (abstract or concrete) and implement interfaces.
- ✨ A class can extend only one abstract class (single inheritance), but it can implement multiple interfaces.

Example: An Abstract `Vehicle` Class

```
public abstract class Vehicle {  
    // Abstract method  
    public abstract void start();  
  
    // Concrete method  
    public void stop() {  
        System.out.println("Vehicle stopped.");  
    }  
}
```

Explain:

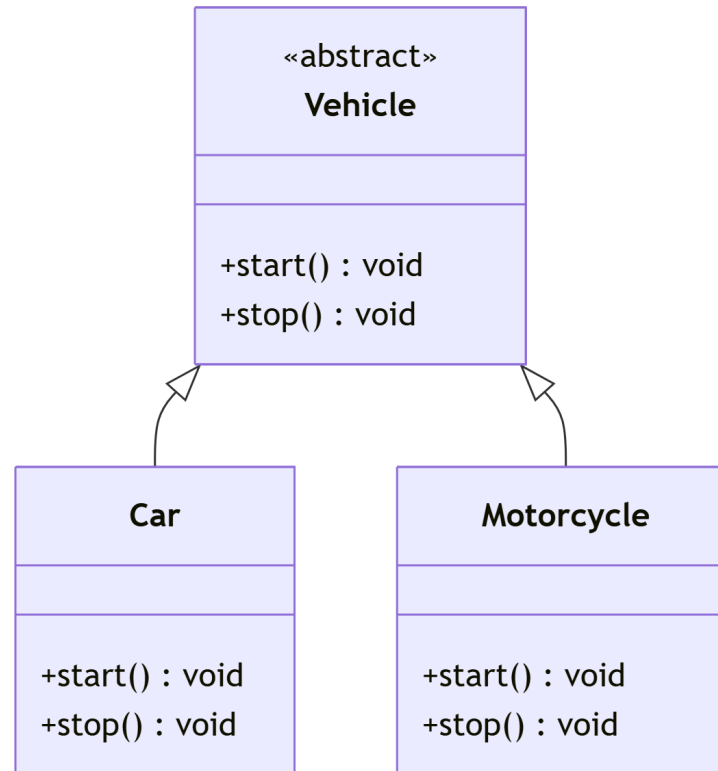
- ✨ The `Vehicle` class is abstract and cannot be instantiated.
- ✨ It contains an abstract method `start()` that must be implemented by any subclass.
- ✨ It also has a concrete method `stop()` that provides default behavior.

Example: Subclassing `Car` from `Vehicle`

```
public class Car extends Vehicle {  
    @Override  
    public void start() {  
        System.out.println("Car started.");  
    }  
}
```

👉 UML Representation

✨ An abstract class has relationships with its subclasses, represented in UML as follows:



👉 Notation in UML

Abstract class name
is italicized



GeometricObject

```
-color: String  
-filled: boolean  
-dateCreated: java.util.Date
```

The # sign indicates
protected modifier



```
#GeometricObject()  
#GeometricObject(color: string, filled:  
    boolean)  
+getColor(): String  
+setColor(color: String): void  
+isFilled(): boolean  
+setFilled(filled: boolean): void  
+getDateCreated(): java.util.Date  
+toString(): String
```

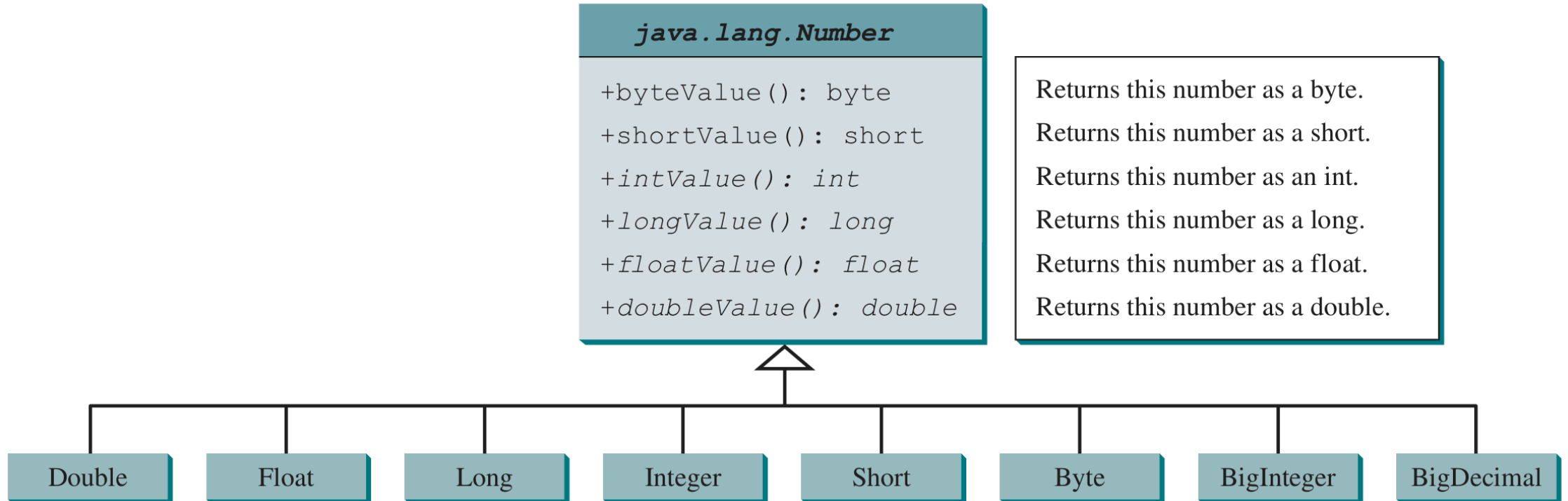
Abstract methods
are italicized



```
+getArea(): double  
+getPerimeter(): double
```


13.3. Case Study: The Abstract Number Class

Practice.



13.4. Case Study: Calendar and GregorianCalendar

Practice.

java.util.Calendar

```
#Calendar()  
+get(field: int): int  
+set(field: int, value: int): void  
+set(year: int, month: int,  
    dayOfMonth: int): void  
+getActualMaximum(field: int): int  
+add(field: int, amount: int): void  
+getTime(): java.util.Date  
  
+setTime(date: java.util.Date): void
```



java.util.GregorianCalendar

```
+GregorianCalendar()  
+GregorianCalendar(year: int,  
    month: int, dayOfMonth: int)  
+GregorianCalendar(year: int,  
    month: int, dayOfMonth: int,  
    hour: int, minute: int, second: int)
```

Constructs a default calendar.

Returns the value of the given calendar field.

Sets the given calendar to the specified value.

Sets the calendar with the specified year, month, and date. The month parameter is 0-based; that is, 0 is for January.

Returns the maximum value that the specified calendar field could have.

Adds or subtracts the specified amount of time to the given calendar field.

Returns a `Date` object representing this calendar's time value (million second offset from the UNIX epoch).

Sets this calendar's time with the given `Date` object.

Constructs a `GregorianCalendar` for the current time.

Constructs a `GregorianCalendar` for the specified year, month, and date.

Constructs a `GregorianCalendar` for the specified year, month, date, hour, minute, and second. The month parameter is 0-based, that is, 0 is for January.

13.5. Interfaces

Motivation

- ✨ In OOP, we often want to ensure that different classes share certain methods, even if they are not related by inheritance.
- ✨ So, **interfaces** let us list methods that a class must have. This helps us write flexible and reusable code.
- ✨ This helps us to distribute tasks and ensure that certain methods are always available in subclasses.

What is an Interface?

- ✨ An **interface** is a reference type in Java, similar to a class, that can contain only constants, method signatures, default methods, static methods, and nested types.
- ✨ It cannot contain instance variables or concrete methods (until Java 8, which introduced default and static methods).
- ✨ A class that implements an interface must provide concrete implementations for all of its abstract methods.

👉 Create an Interface

✨ An **interface** is defined using the `interface` keyword. It can contain abstract methods, default methods, static methods, and constants.

Syntax:

```
public interface InterfaceName {  
    void methodName(); // abstract  
    default void defaultMethod() {} // default (Java 8+)  
    static void staticMethod() {} // static (Java 8+)  
}
```

Explain:

- ✨ `default` : Provides a method implementation in the interface, which classes can override.
- ✨ `static` : Defines a method callable on the interface itself, without an instance.

👉 Using an Interface

✨ To use an interface, a class must implement it using the `implements` keyword and provide concrete implementations for all its abstract methods.

Syntax:

```
public class ClassName implements InterfaceName {  
    // Implement abstract methods  
    @Override  
    public void methodName() {  
        // Implementation code  
    }  
}
```

Interface Types

 **Marker Interface**

 **Regular Interface**

 **Functional Interface**

 **Default Method**

 **Static Method**

 **Nested Interface**

 **Generic Interface**

Marker Interface

✨ The **Marker Interface** is a special type of interface that does not contain any methods or fields.

✨ It is used to indicate that a class possesses a certain property or behavior.

Syntax: Marker Interface

```
public interface MarkerInterface {  
    // No methods, just a marker  
}
```

Regular Interface

✨ A **Regular Interface** is a standard interface that can contain abstract methods, default methods, and static methods.

Syntax: Regular Interface

```
public interface RegularInterface {  
    void methodOne(); // abstract  
    default void methodTwo() {} // default (Java 8+)  
    static void staticMethod() {} // static (Java 8+)  
}
```

Functional Interface

✨ A **Functional Interface** is an interface that contains exactly one abstract method. It can have multiple default or static methods.

Syntax: Functional Interface

```
@FunctionalInterface
public interface FunctionalInterface {
    void singleMethod(); // Single abstract method
    default void defaultMethod() {} // Optional default method
}
```

Nested Interface

✨ A **Nested Interface** is an interface defined within another class or interface. It can be used to logically group interfaces that are only relevant to the outer class.

Syntax: Nested Interface

```
public class OuterClass {  
    public interface NestedInterface {  
        void nestedMethod();  
    }  
}
```

👉 Generic Interface

✨ A **Generic Interface** is an interface that uses type parameters, allowing it to operate on different data types while maintaining type safety.

Syntax: Generic Interface

```
public interface GenericInterface<T> {  
    void genericMethod(T parameter);  
}
```

Note:

✨ A generic interface uses type parameters to allow for flexibility with different data types.

✨ `<T>` is a type parameter that can be replaced with any data type when implementing the interface.

👉 Interface Rules

- ✨ An interface is defined using the `interface` keyword.
- ✨ All methods in an interface are implicitly `public` and `abstract`.
- ✨ Fields in an interface are implicitly `public`, `static`, and `final`.
- ✨ A class can implement multiple interfaces, allowing for a form of multiple inheritance.

Example: Making interfaces with `default` keyword

```
public interface Animal {  
    // Abstract method  
    void sound();  
  
    // Default method  
    default void eat() {  
        System.out.println("Animal is eating.");  
    }  
}
```

Explain:

- ✨ The `Animal` interface has an abstract method `sound()` that must be implemented by any class that implements this interface.
- ✨ It also has a default method `eat()` that provides a default implementation, which can be overridden by implementing classes.

Example: Implementing Multiple Interfaces

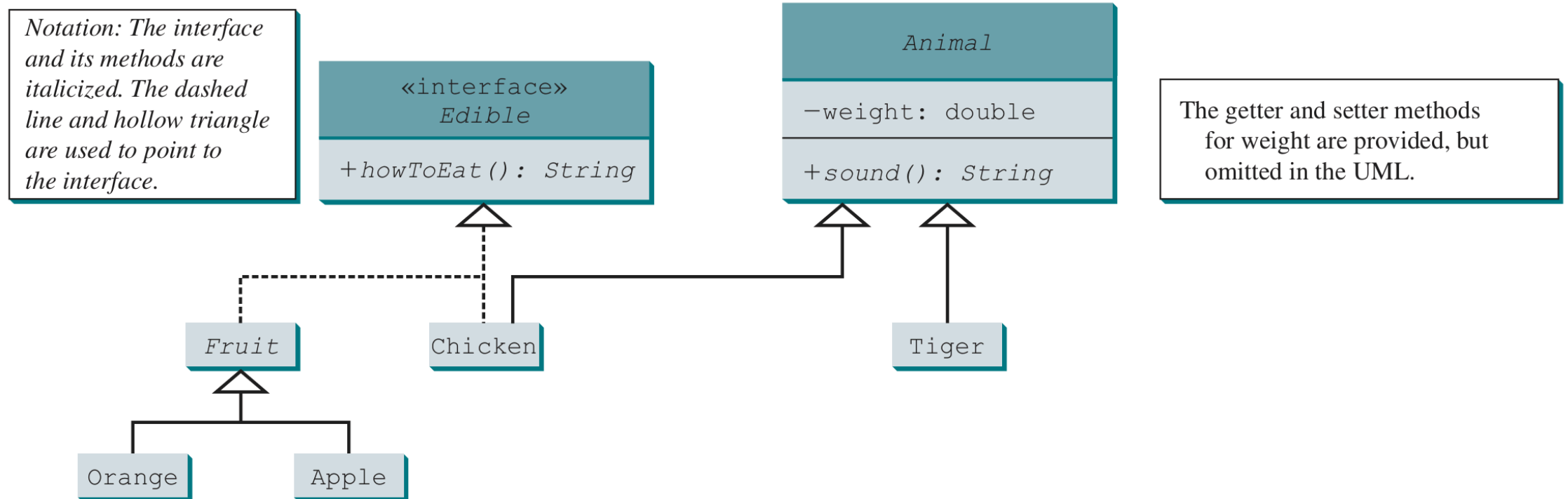
```
interface Flyable { void fly(); }  
interface Swimmable { void swim(); }  
  
class Duck implements Flyable, Swimmable {  
    public void fly() { System.out.println("Duck is flying."); }  
    public void swim() { System.out.println("Duck is swimming."); }  
}
```

Explain:

- ✨ The `Duck` class implements both `Flyable` and `Swimmable` interfaces, providing concrete implementations for the methods defined in those interfaces.
- ✨ This allows the `Duck` class to exhibit both flying and swimming behavior.

👉 Example: Edible Interface

✨ The edible interface is a common example used to demonstrate how interfaces can be used to define behavior that multiple classes can implement.



Example: Edible Interface

```
// Edible interface
interface Edible {
    String howToEat();
}

// Fruit class implementing Edible
abstract class Fruit implements Edible {
    // Common fruit properties could go here
}
```

```
// Orange class
class Orange extends Fruit {
    @Override
    public String howToEat() {
        return "Peel and slice the orange";
    }
}

// Apple class
class Apple extends Fruit {
    @Override
    public String howToEat() {
        return "Wash and bite the apple";
    }
}
```

```
// Chicken class implementing Edible
class Chicken implements Edible {
    @Override
    public String howToEat() {
        return "Cook the chicken thoroughly";
    }
}

// Main class
public class Main {
    public static void main(String[] args) {
        Edible[] edibles = {
            new Orange(),
            new Apple(),
            new Chicken()
        };

        for (Edible item : edibles) {
            System.out.println(item.howToEat());
        }
    }
}
```

End of the Chapter